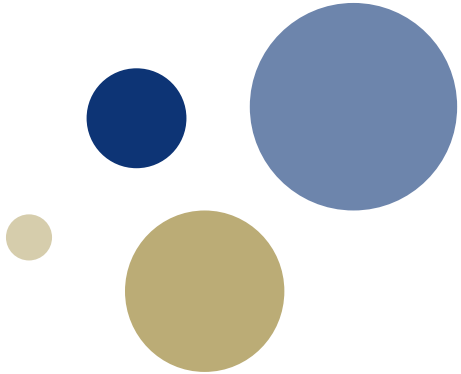
NTNU

Kunnskap for en bedre verden



TDT4110 ITGK – Uke 35

Dag Olav Kjellemo

Institutt for datateknologi og informatikk (IDI)

Fra forrige uke

- Vi har sett på
  - Input og print
  - Aritmetiske uttrykk med +, -, *, /, //, **, %
  - Variable og tilordning
  - Syntaks- og navne-feil

Husk: Les Jupyter notatbøkene, gjør øvingsoppgaver.

Still spørsmål i Canvas og til øvingslærere, diskuter, søk på nett, spør en LLM når, men jobb med svarene du får!

Denne uka

- Vi jobber gjennom deler av notatbøkene
  - Mer om datatyper, inkludert lister og arrays.
  - Konvertering av datatyper
  - Funksjoner – strukturering og gjenbruk av kode
  - Hvordan spore program-flyten (tracing)
  - Tallrepresentasjon – en av mange grunner til å ikke naïvt tro på resultater fra en datamaskin – de fleste reelle tall kan **ikke** representeres **eksakt, dette må vi være obs på!**

Viktige ting å lære denne uka

- Datatyper og bruksområder
  - Enkle datatyper
  - Sammensatte datatyper som lister og arrays
  - Forskjellige operasjoner knyttet til datatypene: Hva kan vi gjøre med dem?

- Flyttallsrepresentasjon og forskjellige typer “feil”
  - Hvordan representeres flytttall
  - Hvorfor kan vi få avrundingsfeil og andre “rare” resultater ved utregninger
  - Noen ting vi kan gjøre for å minske feilene

Enkle (ikke-sammensatte) datatyper

Hva slags data	Kalles	Python-navn i kode	Eksempel
Hele tall	Integer	int	antall = 4
Desimaltall	Floating point	float	høyde = 1.88
Sannhetsverdi	Boolean	bool	ferdig = False itgk = True
Tekst	String	str	navn = «Ole»

Notebook «Datatyper – Intro»,

Operasjoner og Operatorer

- Forskjellige datatyper støtter forskjellige operasjoner:
  - Legge sammen tall (+), konkatenere lister eller tekststrenger (+)
- Python har reserverte symboler, kalt operatorer, som er knyttet til forskjellige operasjoner:
  - Aritmetiske +, -, *, /, //, **, %
  - Sammenligning: ==, !=, <, >, >=, <=
  - Tilordning: =, +=, -=, *=, /=, //=
- Operatorene (symbolene over) kan representere forskjellige operasjoner, avhengig av hvilke type data de opererer på
- Hvis operasjonene gjøres med variabler, så hentes først verdien, før operasjonen gjøres.

Konvertering mellom datatyper

- Vanlig problem: Vi har data av «feil» type for en operasjon
- Vi har funksjoner som kan konvertere mellom typer (der det gir mening)

Funksjon	Forklaring, eksempler	
int()	int(4.7) gir 4, int('44') gir tallet 44	
float()	float(3) gir 3.0, float('3.14') gir 3.14	
str()	Gir tekstrepresentasjon	
bool()	gir False for 0, tom streng, tom liste, True for alt annet	Falsy og Truthy verdier

HUSK: input() returnerer alltid tekst, som må konverteres til den typen en skal ha, f.eks. int.

Sammensatte datatyper

- Vi kan kombinere flere data i en sammensatt datatype
- Det er ett dataobjekt, og lagres i én variabel
- Lister (innebygd i Python), og Numpy arrays

Type data	Term (I kode)	Eksempler
Lister – kan være miks av hva som helst	Liste (list)	primtall = [2,3,5,7] favoritter = [«blå», 7, «Trondheim», [2,3,5]]
Array – alle elementer må ha samme type	Array (numpy.array)	import numpy as np temperaturer = np.array([19.3, 22.4])

Se notebook «Datatyper – Liste og Array»

Bruk av lister og arrays

FELLES:

- De gjør at vi kan håndtere flere data under ett.
- Vi kan hente ut enkelt-elementer eller deler av listen (indeks, slice)
- Vi kan sette sammen eller splitte lister og arrays

LISTER:

- Lister kan lages ved å angi verdier: [1, 2, «tre»]
- Lister kan inneholde en blanding av element-typer

ARRAYS:

- Alle elementer må ha samme datatype
- På numpy-arrays så kan vi gjøre operasjoner på alle elementer i en operasjon.
- Mer effektive for beregninger enn vanlige lister.
- Arrays lages (initialiseres) ofte ved å angi en liste som input:
  - A = np.array([1,2,3])

Heltall (int) og flyttall (float)

- Python kan representere vilkårlig store HELTALL. Dette er alltid nøyaktig
- Vi kan ikke representere alle reelle tall (krever generelt uendelig mange siffer)
- Vi representerer en UTVALG av reelle tall med FLYTTALL (float)
- Det brukes et fast antall siffer pluss OG en eksponent for å angi størrelsesorden
- Dette fordi det er hensiktsmessig å ha lik RELATIV PRESISJON. Avstanden mellom representerbare tall bør være liten for små tall, men vi kan tillate større avstand mellom store tall.

Flyttallrepresentasjon

Bruker 10-tallsystemet for å illustrere konseptet med 3 siffer for mantissa og ett siffer for eksponent.

- For å representere små og store tall, ganger vi med 10er-potenser
- Underflyt: Tallet er for lite til å representeres
- Overflyt: Tallet er for stort til å representeres

I datamaskinen gjøres det på *lignende* måte, men binært

Tall	Flyttallsrepresentasjon
1.23	$1.23 \cdot 10^0$
1230.0	$1.23 \cdot 10^3$
0.00123	$1.23 \cdot 10^{-3}$
1234567.0	$1.23 \cdot 10^6$ (avrunding)
123000000000000000000.0	Inf (Overflyt)
0.00000000000000000123	0 (Underflyt)

Fordeling av representerbare tall

Tallene ligger tettest rundt 0, og så blir det lengre og lengre mellom tallene, jo lenger vekk fra 0 vi er.

Tall	Neste representerbare tall	Differanse mellom tallene
$0.00 \cdot 10^0$	$0.01 \cdot 10^{-9}$	10^{-11} (liten)
$1.00 \cdot 10^0$	$1.01 \cdot 10^0$	10^{-2} (Middels)
$1.0 \cdot 10^9$	$1.01 \cdot 10^9$	10^7 (Stor)

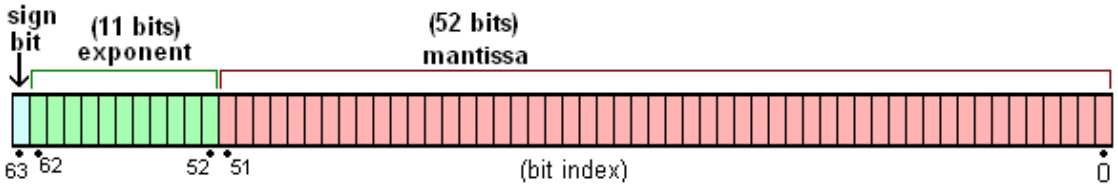
Binær tallrepresentasjon

Den viktigste utfordringen med flyttall, er at ikke alle tall kan representeres eksakt. Et mye mindre problem, men som kan være noe forvirrende, er at:

- Datamaskiner bruker binære tall internt
- Python konverterer mellom binære og desimaltall for input/output.
- Det er ingen problemer knyttet til denne konverteringen for HELTALL
  - Python kan i prinsippet representere alle heltall, og bruker så mange siffer den trenger for hvert tall.

For float (flyttall) er situasjonen annerledes. Der brukes det fast 64 bits.

- En bit til fortegn, 52 bits til sifrene (mantissa), og eksponent.
- Dette gir omtrent 15 desimaler i titallssystemet



Konvertering binær - titallssystem

- Det er ikke pensum å gjøre dette for hånd
- Eksempler med heltall:

Desimal	Binær
1	1
2	10
3	11
4	100
5	101
10	1010

Konvertering binær - titallssytem

- Eksempler for desimaltall (tall som ikke er heltall)

Desimal	Binær
½ = 0.5	0.1
¼ = 0.25	0.01
1/8 = 0.125	0.001
1/5 = 0.2	0.0011001100110011...
1/3 = 0.333...	0.01010101...
π = 3,1415926535 8979323846 264...	11.001...

Konsekvenser

- De fleste *reelle* tall kan **ikke** representeres som datamaskinens flyttall-type ***float***
- Tall avrundes til nærmeste float
- Både relativ og absolutt avrundingsfeil er relevant:
  - Relativ feil: Feilen delt på tallets størrelse (f.eks. angitt som prosent)
- Resultatet av regneoperasjoner må også avrundes, og mange operasjoner etter hverandre kan *akkumulere* feil, dvs. gi større og større feil.
  - Feilen kan forstørres for hver utregning

Eksempel fra eksamen:

Hvilke av disse tallene kan representeres eksakt som enten heltall eller flyttall i Python? For hvert tall, sett kryss i rett kolonne. Ingen minuspoeng for feil, så du BØR svare noe også der du er usikker.

	Eksakt	IKKE eksakt
0.2		
3131313131313		
19		
1.75e-3		
1.6		
-1.25		

Hvordan vise at 1.75e-3 ikke kan representeres eksakt

Et desimaltall kan representeres eksakt hvis tallet kan skrives som en brøk med 2^n i nevner:

$$1.75 \cdot 10^{-3} = \frac{175}{10^5} = \frac{5^2 \cdot 7}{2^5 \cdot 5^5} = \frac{7}{2^5 \cdot 5^3}$$

Som ikke kan forkortes mer, slik at vi ikke kan bli kvitt 5-faktorene i nevneren.

Sammenligne flyttall

- Å sammenligne flyttall for NØYAKTIG likhet er sjeldent det man ønsker å gjøre
- Det vanlige er å angi en toleranse for likhet.
- Hvor stor toleranse, avhenger av situasjonen
- Begge bibliotekene math og numpy har funksjonen **isclose** som sjekker om den relative avstanden er lite

Eksempel:

- 0.1 + 0.1 + 0.1 == 0.3 gir False
- math.isclose(0.1+0.1+0.1, 0.3) gir True

Avrunding, Relativ og absolutt feil

Hvis x er en verdi som vi tilnærmer med $\tilde{x}$,

Absolutt feil: $|x - \tilde{x}|$

Relativ feil: $\frac{|x-\tilde{x}|}{x}$ (for $x = 0$ virker ikke dette)

Når vi representerer x med $\tilde{x} = fl(x)$ (nærmeste float), så vil den *relative feilen* begrenset av ca 1.1e-16. Dette er *uavhengig* av hvor stor x er.

Den absolutte feilen blir begrenset av ca. $1.1e{-}16 \cdot |x|$, som blir mindre og mindre med $|x|$.

Eksempel

Vi vet at $\sin(\pi) = 0$.

Hva skjer hvis vi prøver å regne ut dette i Python?

Får vi 0?

Eksempler på problemer

- Summere / subtrahere tall av svært ulik størrelse
  - Minste tall mister presisjon (bakerste desimaler), eller også
  - a + b = a (b «drukner» i a)
- Subtrahere to nesten like store tall
  - Mister presisjon, får færre (signifikante/sikre) siffer i resultat
- Gange sammen to veldig store tall, eller veldig små tall
  - Overflyt: Resultatet blir for stort, Underflyt: Blir rundet av til 0
- Størrelser som matematisk er like, er **ikke** like *numerisk*:
  - a + b - a kan bli forskjellig fra b, pga avrundinger underveis

Overflyt og underflyt

- Største float er ca $1.79 \cdot 10^{308}$
  - Større verdier gir overflyt: **inf** (infinity, uendelig, men er jo ikke det…)
- Minste positive float (nærmest null): ca $2.2 \cdot 10^{-308}$
  - Hvis mindre: Underflyt, verdi blir 0.0

Valg av rekkefølge på operasjoner kan påvirke resultat.

- En «tommelfingerregel» er å holde mellomregninger nær 1,
- Dette omskriving av uttrykk som matematisk sett gir det samme, men som numerisk blir forskjellig
- Eksempel: Pythagoras:

Eksempel: Pythagoras, lengde til hypotenusen

Vanlig formel

$$h = \sqrt{a^2 + b^2}$$

For å unngå overflyt for veldig store tall:

Hvis $|a| \geq |b|$, kan en bruke

$$h = |a| \sqrt{1 + \left(\frac{b}{a}\right)^2}$$

Hvis $|a| \leq |b|$, så bytt om på a og b i formelen.

Eksempel: Andregradsformel

Vanlig formel

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Hvilke mulige problemer her?

Hvilke forbedringer?

Multiplikasjon og divisjon

For multiplikasjon og divisjon så har vi færre problemer enn med addisjon eller subtraksjon for *relativ nøyaktighet.*

F.eks. hvis vi jobbet med 3 desimaler, så vil ikke størrelsesordenen på tallet påvirke den relative nøyaktigheten:

- 1.23*1.24 =1.5252, som blir avrundet til 1.53.
- 123*0.124 = 15.252 som bli avrundet til 15.3

Ha i bakhode følgende råd

I dette faget skal vi lære om programmering, og har litt om beregninger.

Generelt er det vanskelig å implementere numeriske algorimer på en god måte.

For *viktig* arbeid er det lurt å finne ferdig kode en kan gjenbruke for f.eks. å løse likninger.

Oppsummering

- Datatyper
  - Int, float, bool, string, list, np.array
  - Konvertere mellom datatyper når det trengs
- Vi må regne med at regning med flyttall IKKE ER EKSAKT
  - Det trenger ikke være et stort problem, men kan også være KATASTROFALT
  - Det er kan være vanskelig å være sikker på at beregninger er GODE NOK